

```

module traffic_controller (
    input clk,
    input rst,
    output reg [1:0] lights_A, // 00 = RED, 01 = YELLOW, 10 = GREEN
    output reg [1:0] lights_B
);

// State encoding
parameter A_GREEN_STATE = 2'b00;
parameter A_YELLOW_STATE = 2'b01;
parameter B_GREEN_STATE = 2'b10;
parameter B_YELLOW_STATE = 2'b11;

reg [1:0] current_state, next_state;
reg [3:0] counter; // 4-bit counter: enough for 0-15

// State durations in clock cycles (assuming 1 clock = 1s for
example)
parameter GREEN_DURATION = 4'd10;
parameter YELLOW_DURATION = 4'd3;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        current_state <= A_GREEN_STATE;
        counter <= 0;
    end else begin
        if (counter == (current_state == A_YELLOW_STATE ||
current_state == B_YELLOW_STATE ? YELLOW_DURATION - 1 :
GREEN_DURATION - 1)) begin
            current_state <= next_state;
            counter <= 0;
        end else begin
            counter <= counter + 1;
        end
    end
end
end

```

```

// Next state logic
always @(*) begin
    case (current_state)
        A_GREEN_STATE:    next_state = A_YELLOW_STATE;
        A_YELLOW_STATE:   next_state = B_GREEN_STATE;
        B_GREEN_STATE:    next_state = B_YELLOW_STATE;
        B_YELLOW_STATE:   next_state = A_GREEN_STATE;
        default:          next_state = A_GREEN_STATE;
    endcase
end

// Output logic
always @(*) begin
    case (current_state)
        A_GREEN_STATE: begin
            lights_A = 2'b10; // GREEN
            lights_B = 2'b00; // RED
        end
        A_YELLOW_STATE: begin
            lights_A = 2'b01; // YELLOW
            lights_B = 2'b00; // RED
        end
        B_GREEN_STATE: begin
            lights_A = 2'b00; // RED
            lights_B = 2'b10; // GREEN
        end
        B_YELLOW_STATE: begin
            lights_A = 2'b00; // RED
            lights_B = 2'b01; // YELLOW
        end
        default: begin
            lights_A = 2'b00;
            lights_B = 2'b00;
        end
    endcase
end

```

endmodule